Agent Orchestrator – User Guide

Agent Orchestrator Team

2026-04-01

Agent Orchestrator

User Guide

Version 1.0

April 1, 2026

Table of contents

1 Introduction **4**

2 Getting Started **4**
2.1 Prerequisites . . . 4
2.2 Installation Options . . . 4
2.2.1 Lite Mode (pip install) . . . 4
2.2.2 Standard Mode (Docker Compose) . . . 5
2.3 Initializing a Workspace . . . 5
2.4 Starting the Server . . . 6

3 Configuration Model **6**
3.1 agents.yaml . . . 6
3.2 workflow.yaml . . . 7
3.3 governance.yaml . . . 8
3.4 workitems.yaml (Optional) . . . 9

4 Agents **9**
4.1 Creating Agents . . . 9
4.1.1 Agent Properties . . . 9
4.1.2 System Prompts . . . 10
4.1.3 LLM Configuration . . . 10
4.1.4 Concurrency and Retry Policies . . . 10
4.2 Managing Agents at Runtime . . . 11

5 Workflow and Phases **11**
5.1 Phase Pipeline Model . . . 11
5.1.1 Phase Properties . . . 11
5.1.2 Phase Transitions . . . 12
5.1.3 Quality Gates . . . 12
5.2 Execution Model . . . 12

6 Work Items **12**
6.1 Submitting Work Items . . . 12
6.1.1 API Submission . . . 12
6.1.2 CLI Submission . . . 13
6.2 Work Item Lifecycle . . . 13
6.3 Priority and SLA . . . 13
6.4 Tracking Work Items . . . 13

7 Governance **14**
7.1 Delegated Authority Model . . . 14
7.2 Policy Configuration . . . 14
7.2.1 Rule Types . . . 14
7.2.2 Actions . . . 14
7.3 Review Queue . . . 15

8 Audit Trail **15**

8.1 Hash-Chained Audit Log . . . 15
8.2 Record Structure . . . 15
8.3 Record Types . . . 16
8.4 Querying the Audit Trail . . . 16

9 Knowledge Store **16**
9.1 Content-Addressable Storage . . . 16
9.2 Semantic Search . . . 16
9.3 Auto-Extraction . . . 17

10 Connectors **17**
10.1 Capability-Based Model . . . 17
10.2 Capability Types . . . 17
10.3 Built-in Providers . . . 17
10.4 MCP Bridge . . . 18
10.5 Connector Permissions . . . 18
10.6 Connector Tracing . . . 18

11 Evaluation and Simulation **19**
11.1 LLM-as-Judge . . . 19
11.2 A/B Testing . . . 19
11.3 Benchmarks . . . 19
11.4 Simulation Sandbox . . . 19

12 API Reference **20**
12.1 Endpoint Groups . . . 20
12.2 Common Response Patterns . . . 21

13 CLI Reference **21**
13.1 Commands . . . 21
13.2 Global Options . . . 22

14 Built-in Profiles **22**
14.1 content-moderation . . . 22
14.2 software-dev . . . 22
14.3 research-team . . . 23

15 Common Workflows **23**
15.1 Workflow 1: Basic Work Item Processing . . . 23
15.2 Workflow 2: Custom Multi-Agent Pipeline . . . 23
15.3 Workflow 3: Evaluation and Optimization . . . 23
15.4 Workflow 4: Adding External Connectors . . . 24

16 Troubleshooting **24**
16.1 Server Does Not Start . . . 24
16.2 LLM Calls Failing . . . 24
16.3 Work Items Stuck in Pending . . . 24
16.4 Connector Invocations Failing . . . 25
16.5 Hash Chain Verification Fails . . . 25

16.6 Docker Compose Issues 25

1 Introduction

Agent Orchestrator is a domain-agnostic platform for orchestrating multi-agent AI workflows. It provides built-in governance, auditing, observability, and connector integration without encoding any domain-specific logic. All domain knowledge lives in YAML configuration profiles, meaning the same platform can power content moderation pipelines, software development workflows, security incident response, research teams, or any other multi-agent use case.

The platform coordinates multiple AI agents through phase-based pipelines, where each phase can run agents in parallel or sequentially. A non-blocking governance layer evaluates policies and enforces delegated authority thresholds. Every decision, agent action, phase transition, and connector invocation is recorded in a hash-chained, tamper-evident audit trail.

Key platform capabilities include:

- Multi-agent orchestration with configurable phase pipelines
- Support for 5 LLM providers: OpenAI, Anthropic, Google, Grok, and Ollama
- Non-blocking governance with delegated authority and human review queues
- Hash-chained JSONL audit trail for tamper-evident logging
- Knowledge store with content-addressable storage and semantic search
- Connector framework with 10 capability types and 12+ providers (including MCP bridge)
- LLM-as-judge evaluation, A/B testing, benchmarks, and simulation sandbox
- 40+ REST API endpoints across 20 route groups
- CLI for workspace management, server operation, and work submission
- Docker deployment with PostgreSQL for production environments

2 Getting Started

2.1 Prerequisites

You need at least one of the following LLM provider API keys, or a running Ollama instance:

Provider	Environment Variable	Notes
OpenAI	`AGENT_ORCH_OPENAI_API_KEY`	GPT-4, GPT-4o models
Anthropic	`AGENT_ORCH_ANTHROPIC_API_KEY`	Claude models
Google	`AGENT_ORCH_GOOGLE_API_KEY`	Gemini models
Grok	`AGENT_ORCH_GROK_API_KEY`	xAI Grok models
Ollama	No key needed	Local models at `http://localhost:11434`

2.2 Installation Options

2.2.1 Lite Mode (pip install)

For local development, prototyping, and single-developer projects:

```python
# Core only
pip install agent-orchestrator

# With all LLM provider SDKs
pip install "agent-orchestrator[llm]"

# With MCP support
pip install "agent-orchestrator[mcp]"

# Full development install
pip install "agent-orchestrator[llm,mcp,dev]"
```

2.2.2 Standard Mode (Docker Compose)

For production deployments with PostgreSQL:

```python
git clone <your-repo-url> agent-orchestrator
cd agent-orchestrator
cp .env.example .env
# Edit .env with your API keys and database credentials
docker compose up -d
```

This starts three services:

Service	Description	Port
`api`	FastAPI server with all LLM SDKs	8000
`db`	PostgreSQL 16 database	5432
`ollama`	Local LLM inference (optional)	11434

2.3 Initializing a Workspace

A workspace is a directory containing your YAML configuration profiles. Initialize one with the CLI:

```python
# Empty workspace
agent-orchestrator init my-workspace

# From a built-in template
agent-orchestrator init --template content-moderation my-workspace
agent-orchestrator init --template software-dev my-workspace
agent-orchestrator init --template research-team my-workspace
```

A workspace directory contains:

```
my-workspace/
  settings.yaml          # API keys, endpoints, deployment mode
  profiles/
```

```
default/
  agents.yaml        # Agent definitions
  workflow.yaml       # Phase pipeline
  governance.yaml     # Policies and thresholds
  workitems.yaml      # Work item templates (optional)
  mcp.yaml            # MCP server configs (optional)
```

2.4 Starting the Server

```
# Validate configuration first
agent-orchestrator validate my-workspace

# Start the API server on port 8000
agent-orchestrator serve my-workspace
```

Verify the server is running:

```
curl http://localhost:8000/api/v1/health
# {"status": "ok", "version": "0.6.0", ...}
```

Interactive API documentation is available at `http://localhost:8000/docs`.

3 Configuration Model

All domain logic in Agent Orchestrator is expressed through four YAML configuration files. The platform core contains zero hardcoded domain knowledge.

3.1 agents.yaml

Defines the AI agents that participate in your workflow. Each agent has an identity, system prompt, LLM configuration, skills, and runtime policies.

```yaml
agents:
  - id: "analyst"
    name: "Data Analyst"
    role: "analyst"
    system_prompt: |
      You are a data analyst. Examine the submitted data,
      identify patterns, anomalies, and key insights.
      Produce a structured findings report.
    llm:
      provider: "openai"
      model: "gpt-4o"
      temperature: 0.3
      max_tokens: 4096
    skills:
      - "data_analysis"
```

```yaml
      - "report_generation"
    concurrency: 3
    retry_policy:
      max_retries: 2
      backoff_base: 1.0
      backoff_factor: 2.0
    connectors:
      - capability: "documents"
        operations: ["retrieve"]
      - capability: "search"
        operations: ["query"]

  - id: "reviewer"
    name: "Quality Reviewer"
    role: "reviewer"
    system_prompt: |
      You are a quality reviewer. Evaluate the analyst's
      findings for accuracy, completeness, and clarity.
      Provide a quality score and improvement suggestions.
    llm:
      provider: "anthropic"
      model: "claude-sonnet-4-20250514"
      temperature: 0.2
      max_tokens: 2048
    concurrency: 1
```

3.2 workflow.yaml

Defines the phase pipeline that work items traverse. Each phase specifies which agents execute, whether they run in parallel, and what transitions occur on success or failure.

```yaml
workflow:
  initial_phase: "analysis"
  phases:
    - id: "analysis"
      name: "Data Analysis"
      agents: ["analyst"]
      parallel_execution: false
      on_success: "review"
      on_failure: "failed"
      quality_gate:
        min_confidence: 0.7

    - id: "review"
      name: "Quality Review"
      agents: ["reviewer"]
```

```yaml
      parallel_execution: false
      on_success: "complete"
      on_failure: "analysis"

    - id: "complete"
      name: "Complete"
      terminal: true

    - id: "failed"
      name: "Failed"
      terminal: true
```

3.3 governance.yaml

Defines policies, delegated authority thresholds, and escalation rules. The governance layer is non-blocking – it evaluates policies and logs decisions without halting the pipeline, unless a policy explicitly requires human approval.

```yaml
governance:
  delegated_authority:
    auto_approve_threshold: 0.8
    escalation_threshold: 0.5
    human_review_below: 0.5

  policies:
    - id: "quality-check"
      name: "Minimum Quality Score"
      description: "Ensure outputs meet minimum quality threshold"
      rule_type: "threshold"
      parameters:
        field: "quality_score"
        operator: ">="
        value: 0.7
      action: "flag"

    - id: "cost-limit"
      name: "Per-Item Cost Limit"
      description: "Escalate if estimated cost exceeds budget"
      rule_type: "threshold"
      parameters:
        field: "estimated_cost"
        operator: "<="
        value: 5.00
      action: "escalate"

    - id: "sensitive-content"
      name: "Sensitive Content Review"
```

```yaml
      description: "Require human review for flagged content"
      rule_type: "keyword_match"
      parameters:
        fields: ["content", "summary"]
        keywords: ["confidential", "restricted"]
      action: "require_human_review"
```

3.4 workitems.yaml (Optional)

Defines templates and defaults for work items submitted to the engine. Work items can also be submitted directly via the API or CLI without a template.

```yaml
workitems:
  default_priority: 5
  default_sla_minutes: 60
  templates:
    - id: "standard-analysis"
      name: "Standard Data Analysis"
      priority: 5
      metadata:
        type: "analysis"
        category: "standard"
```

4 Agents

4.1 Creating Agents

Agents are defined in `agents.yaml` within your profile directory. Each agent requires at minimum an `id`, `system_prompt`, and `llm` configuration. Agents can also be created and managed at runtime through the REST API or CLI.

4.1.1 Agent Properties

Property	Required	Description
`id`	Yes	Unique identifier (lowercase, hyphens allowed)
`name`	Yes	Human-readable display name
`role`	Yes	Functional role (e.g., analyst, reviewer, coordinator)
`system_prompt`	Yes	Instructions that define the agent's behavior
`llm`	Yes	LLM provider and model configuration
`skills`	No	List of skill identifiers the agent can use
`concurrency`	No	Max parallel executions (default: 1)

Property	Required	Description
`retry_policy`	No	Retry behavior on LLM call failure
`connectors`	No	External system access permissions

4.1.2 System Prompts

The system prompt is the primary mechanism for encoding domain knowledge into an agent. Write prompts that are specific, structured, and include the expected output format.

Effective system prompts should:

- State the agent's role and objective clearly
- Describe the expected input format
- Specify the required output structure
- Include quality criteria the agent should meet
- List any constraints or boundaries

4.1.3 LLM Configuration

Each agent specifies its LLM provider and model independently. Different agents in the same workflow can use different providers.

```yaml
llm:
  provider: "openai"        # openai | anthropic | google | grok | ollama
  model: "gpt-4o"           # Provider-specific model identifier
  temperature: 0.3          # 0.0 = deterministic, 1.0 = creative
  max_tokens: 4096          # Maximum response length
```

Provider	Example Models	Notes
`openai`	`gpt-4o`, `gpt-4-turbo`	Requires `AGENT_ORCH_OPENAI_API_KEY`
`anthropic`	`claude-sonnet-4-20250514`, `claude-opus-4-20250514`	Requires `AGENT_ORCH_ANTHROPIC_API_KEY`
`google`	`gemini-2.0-flash`, `gemini-2.5-pro`	Requires `AGENT_ORCH_GOOGLE_API_KEY`
`grok`	`grok-3`, `grok-3-mini`	Requires `AGENT_ORCH_GROK_API_KEY`
`ollama`	`llama3`, `mistral`, `codellama`	Requires running Ollama at configured endpoint

4.1.4 Concurrency and Retry Policies

The `concurrency` setting controls how many parallel instances of an agent can execute simultaneously. The `AgentPool` enforces this limit with per-agent semaphores.

The retry policy controls behavior when an LLM call fails:

```yaml
retry_policy:
  max_retries: 3          # Number of retry attempts
  backoff_base: 1.0       # Initial delay in seconds
  backoff_factor: 2.0     # Multiplier per retry (exponential backoff)
```

With the above configuration, retries occur at 1s, 2s, and 4s delays.

4.2 Managing Agents at Runtime

Agents support full CRUD operations via the API:

Operation	Endpoint	Method
List agents	`/api/v1/agents`	GET
Get agent	`/api/v1/agents/{id}`	GET
Create agent	`/api/v1/agents`	POST
Update agent	`/api/v1/agents/{id}`	PUT
Delete agent	`/api/v1/agents/{id}`	DELETE
Import agents	`/api/v1/agents/import`	POST
Export agents	`/api/v1/agents/export`	GET

The `AgentPool` supports dynamic scaling at runtime via the `scale_agent()` method, allowing concurrency adjustments without restarting the engine.

5 Workflow and Phases

5.1 Phase Pipeline Model

A workflow is a directed acyclic graph (DAG) of phases. Each phase defines which agents participate, how they execute, and where the pipeline transitions next. Work items enter at the `initial_phase` and traverse the graph until they reach a terminal phase.

5.1.1 Phase Properties

Property	Required	Description
`id`	Yes	Unique phase identifier
`name`	Yes	Human-readable phase name
`agents`	Yes	List of agent IDs that execute in this phase
`parallel_execution`	No	If `true`, agents run concurrently; if `false`, sequentially (default: `false`)
`on_success`	No	Phase ID to transition to on success
`on_failure`	No	Phase ID to transition to on failure
`terminal`	No	If `true`, this is a final phase (no transitions out)
`quality_gate`	No	Conditions that must be met to proceed

5.1.2 Phase Transitions

Transitions are binary: success or failure. The `PipelineManager` handles all graph traversal, locking, and phase history tracking.

```
analysis --[success]--> review --[success]--> complete
    |                       |
    +--[failure]--> failed +--[failure]--> analysis (retry)
```

Key rules:

- A phase succeeds if all its agents complete without unhandled exceptions
- A phase fails if any agent raises an unrecoverable error
- A failure transition can point back to an earlier phase for retry loops (though the graph remains a DAG in structure)
- Terminal phases have no outbound transitions

5.1.3 Quality Gates

Quality gates add conditions beyond simple success/failure. When a quality gate is defined on a phase, the gate conditions must be met for the `on_success` transition to fire. If the gate fails, the `on_failure` transition fires instead.

```yaml
quality_gate:
  min_confidence: 0.7      # Minimum confidence score from agent output
```

5.2 Execution Model

Within a phase, the `PhaseExecutor` manages agent execution:

- **Sequential** (`parallel_execution: false`): Agents execute one at a time in the order listed. Each agent receives the output of the previous agent as additional context.
- **Parallel** (`parallel_execution: true`): All agents execute simultaneously via the `AgentPool`. Results are collected and merged when all agents complete.

The `AgentPool` enforces per-agent concurrency limits using semaphores, preventing any single agent from consuming excessive resources.

6 Work Items

6.1 Submitting Work Items

Work items are the unit of work that flows through the pipeline. Submit them via the API, CLI, or programmatically through the engine.

6.1.1 API Submission

```bash
curl -X POST http://localhost:8000/api/v1/workitems \
  -H "Content-Type: application/json" \
  -d '{
```

```
    "title": "Analyze Q4 sales data",
    "content": "Review the attached Q4 sales figures...",
    "priority": 5,
    "metadata": {
      "department": "sales",
      "quarter": "Q4-2025"
    }
  }'
```

6.1.2 CLI Submission

```
agent-orchestrator submit my-workspace \
  --title "Analyze Q4 sales data" \
  --content "Review the attached Q4 sales figures..." \
  --priority 5
```

6.2 Work Item Lifecycle

Work items progress through the following statuses:

Status	Description
`pending`	Submitted and queued, waiting for processing
`in_progress`	Currently being processed through the pipeline
`paused`	Processing suspended (manual or policy-driven)
`completed`	Successfully reached a terminal success phase
`failed`	Reached a terminal failure phase or encountered an unrecoverable error
`cancelled`	Manually cancelled by a user or the system

6.3 Priority and SLA

Work items are processed in priority order by the `WorkQueue` (implemented as a heap with O(log n) insert). Lower priority numbers are processed first (priority 1 is highest).

Property	Description
`priority`	Integer priority (1 = highest, default: 5)
`sla_minutes`	Target completion time in minutes (for monitoring)
`metadata`	Arbitrary key-value pairs for domain-specific data

6.4 Tracking Work Items

Operation	Endpoint	Method
List work items	`/api/v1/workitems`	GET
Get work item	`/api/v1/workitems/{id}`	GET

Operation	Endpoint	Method
Cancel work item	`/api/v1/workitems/{id}/cancel`	POST

7 Governance

7.1 Delegated Authority Model

The governance framework uses a delegated authority model with three tiers based on confidence scores:

Tier	Confidence Range	Behavior
Auto-approve	>= `auto_approve_threshold`	Decision proceeds without intervention
Escalate	>= `escalation_threshold` and < `auto_approve_threshold`	Decision is flagged and logged, but processing continues
Human review	< `human_review_below`	Decision is placed in the review queue for human approval

This model is non-blocking by default. Policy evaluation and audit logging occur asynchronously and do not halt the pipeline unless a policy explicitly requires human review.

7.2 Policy Configuration

Policies are rules evaluated against work item data, agent outputs, or system metrics. Each policy specifies a rule type, parameters, and an action.

7.2.1 Rule Types

Rule Type	Description	Parameters
`threshold`	Compare a numeric field against a value	`field`, `operator` (`>=`, `<=`, `>`, `<`, `==`), `value`
`keyword_match`	Check if fields contain specific keywords	`fields`, `keywords`
`pattern`	Regex pattern matching against a field	`field`, `pattern`

7.2.2 Actions

Action	Behavior
`flag`	Log the policy match in the audit trail
`escalate`	Log and notify, but continue processing
`require_human_review`	Pause processing and enqueue for human approval

Action	Behavior
`reject`	Reject the work item

7.3 Review Queue

When a policy triggers `require_human_review`, the work item is placed in the `ReviewQueue`. The review queue is a persistent store of pending decisions that require human intervention.

Operation	Endpoint	Method
List pending reviews	`/api/v1/governance/reviews`	GET
Approve review	`/api/v1/governance/reviews/{id}/approve`	POST
Reject review	`/api/v1/governance/reviews/{id}/reject`	POST
Get policy list	`/api/v1/governance/policies`	GET
Create policy	`/api/v1/governance/policies`	POST
Update policy	`/api/v1/governance/policies/{id}`	PUT
Delete policy	`/api/v1/governance/policies/{id}`	DELETE

8 Audit Trail

8.1 Hash-Chained Audit Log

Every significant action in the platform is recorded in an append-only, hash-chained JSONL audit trail. Each record includes a SHA-256 hash of the previous record, creating a tamper-evident chain. If any record is modified or deleted, the hash chain breaks and the tampering is detectable.

8.2 Record Structure

Each audit record contains:

Field	Description
`record_id`	Unique identifier for this record
`sequence`	Monotonically increasing sequence number
`timestamp`	ISO 8601 timestamp
`record_type`	Category of the event (see below)
`work_item_id`	Associated work item (if applicable)
`agent_id`	Associated agent (if applicable)
`phase_id`	Associated phase (if applicable)
`app_id`	Application context identifier
`run_id`	Execution run identifier
`action`	Specific action performed
`summary`	Human-readable description
`data`	Structured payload with event-specific details
`record_hash`	SHA-256 hash (truncated to 16 characters)
`prev_hash`	Hash of the previous record in the chain

8.3 Record Types

Record Type	What Gets Logged
`GOVERNANCE_DECISION`	Policy evaluation results, approval/rejection decisions
`PHASE_TRANSITION`	Phase start, completion, failure, and transitions
`AGENT_ACTION`	Agent execution start, completion, LLM calls
`SYSTEM_EVENT`	Engine start/stop, configuration changes
`WORK_ITEM_CHANGE`	Work item creation, status changes, cancellation
`CONNECTOR_INVOCATION`	External system calls via the connector framework
`MCP_INVOCATION`	Model Context Protocol tool calls

8.4 Querying the Audit Trail

Operation	Endpoint	Method
Query audit records	`/api/v1/audit`	GET
Get audit record	`/api/v1/audit/{id}`	GET
Verify chain integrity	`/api/v1/audit/verify`	GET

Query parameters support filtering by `record_type`, `work_item_id`, `agent_id`, `phase_id`, `app_id`, and time ranges.

9 Knowledge Store

9.1 Content-Addressable Storage

The knowledge store provides persistent, content-addressable storage for artifacts produced during workflow execution. Each artifact is stored with a content-based hash, ensuring deduplication and enabling provenance tracking.

Artifacts can include:

- Agent outputs and intermediate results
- Connector responses and external data
- Evaluation results and quality assessments
- Domain-specific documents and reports

9.2 Semantic Search

The knowledge store supports semantic search over stored artifacts, enabling agents to find relevant prior work, reference materials, and contextual information during execution.

Semantic search uses embedding-based similarity matching. When an agent or user queries the knowledge store, the query is embedded and compared against stored artifact embeddings to return the most relevant results.

9.3 Auto-Extraction

During workflow execution, the platform can automatically extract and store key artifacts from agent outputs. Auto-extraction rules are configurable per agent and per phase, allowing fine-grained control over what gets persisted.

10 Connectors

10.1 Capability-Based Model

Connectors provide a governed abstraction for accessing external systems. The framework uses a capability-based model where agents request a capability type (e.g., "search", "messaging") rather than a specific provider. This allows the underlying provider to be swapped without changing agent configuration.

10.2 Capability Types

Capability Type	Description	Example Providers
`search`	Full-text or semantic search	Tavily, SerpAPI, Brave
`documents`	Document retrieval and management	Confluence
`messaging`	Send/receive messages	Slack, Teams, Email
`ticketing`	Issue and ticket management	Jira, Linear
`repository`	Source code repositories	GitHub, GitLab
`telemetry`	Metrics, logs, traces	Datadog, Splunk, Grafana
`identity`	User and group lookup	LDAP, Okta, Azure AD
`external_api`	Generic HTTP/REST endpoints	Any REST API
`file_storage`	Object and blob storage	S3, GCS, Azure Blob
`workflow_action`	Trigger external workflow steps	CI/CD, webhooks

10.3 Built-in Providers

The platform ships with 11 built-in providers:

Provider	Capability Type	Operations
Tavily	`search`	`query`
SerpAPI	`search`	`query`
Brave	`search`	`query`
Confluence	`documents`	`retrieve`, `search`
Slack	`messaging`	`send`, `receive`
Teams	`messaging`	`send`, `receive`
Email	`messaging`	`send`
Jira	`ticketing`	`create`, `update`, `query`

Provider	Capability Type	Operations
Linear	`ticketing`	`create`, `update`, `query`
GitHub	`repository`	`clone`, `read`, `search`
GitLab	`repository`	`clone`, `read`, `search`

10.4 MCP Bridge

The Model Context Protocol (MCP) bridge enables agents to consume tools from external MCP servers and exposes platform capabilities to external AI clients.

MCP Client mode: Agents call tools hosted on external MCP servers (e.g., GitHub MCP server, filesystem MCP server). Every MCP tool call flows through the `ConnectorService.execute()` pipeline, receiving permission checks, contract validation, and audit logging.

MCP Server mode: The platform exposes its governed capabilities as MCP tools that external AI clients (Claude Desktop, Cursor, etc.) can invoke.

Configure MCP servers in `mcp.yaml` within your profile directory:

```yaml
client:
  servers:
    - server_id: github
      display_name: GitHub MCP Server
      transport: stdio
      command: npx
      args: ["-y", "@modelcontextprotocol/server-github"]
      env:
        GITHUB_PERSONAL_ACCESS_TOKEN: "${GITHUB_TOKEN}"
      capability_type_override: repository
```

Install MCP support: `pip install "agent-orchestrator[mcp]"`

10.5 Connector Permissions

Connectors use deny-by-default permissions. Each agent must explicitly declare the capabilities and operations it is allowed to use in its `connectors` configuration. The `ConnectorGovernanceService` evaluates permissions per-invocation.

10.6 Connector Tracing

Every connector invocation is recorded in the `ConnectorTraceStore`, a ring-buffer trace store that maintains recent execution history. Traces include request parameters, response summaries, latency, and cost metrics.

Operation	Endpoint	Method
List capabilities	`/api/v1/connectors/capabilities`	GET
List providers	`/api/v1/connectors/providers`	GET
Execute connector	`/api/v1/connectors/execute`	POST

Operation	Endpoint	Method
Get traces	`/api/v1/connectors/traces`	GET
Connector governance	`/api/v1/connectors/governance`	GET
Discover providers	`/api/v1/connectors/discovery`	GET

11 Evaluation and Simulation

11.1 LLM-as-Judge

The evaluation framework uses LLM-as-judge to assess agent output quality. A designated evaluator LLM reviews agent outputs against configurable criteria and produces structured quality scores.

Evaluation criteria are configurable per agent and per phase:

- **Accuracy**: How factually correct is the output?
- **Completeness**: Does the output address all aspects of the task?
- **Clarity**: Is the output well-structured and understandable?
- **Relevance**: Does the output stay on topic?

Evaluation results are stored in the knowledge store and recorded in the audit trail.

11.2 A/B Testing

A/B testing enables comparison of different agent configurations, LLM models, or prompt variations against the same work items. The platform supports:

- Creating experiment definitions with control and variant groups
- Routing work items to different configurations
- Collecting quality metrics per variant
- Statistical comparison of results

11.3 Benchmarks

The benchmark framework measures agent performance against curated datasets with known-good outputs. Benchmarks produce metrics including:

- Response quality scores
- Latency percentiles
- Token usage and cost
- Pass/fail rates against quality thresholds

11.4 Simulation Sandbox

The simulation sandbox provides a safe environment for testing workflows without executing real LLM calls or connector invocations. In sandbox mode:

- LLM calls return configurable mock responses
- Connector calls are intercepted and logged but not executed
- The full governance and audit pipeline still runs
- Work items traverse the complete phase graph

This enables workflow validation, governance policy testing, and cost estimation without incurring LLM or API costs.

12 API Reference

The REST API is served at `http://localhost:8000` with all routes under the `/api/v1` prefix. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` (ReDoc).

12.1 Endpoint Groups

Group	Prefix	Endpoints	Description
Health	`/health`	3	Liveness, readiness, and general health checks
Context	`/context`	1	Execution context (app_id, deployment mode)
Agents	`/agents`	7	Full CRUD, import/export for agent definitions
Workflow	`/workflow`	3	Phase listing, phase detail, workflow summary
Work Items	`/workitems`	4	Submit, list, get, cancel work items
Governance	`/governance`	7	Policy CRUD, review queue management
Execution	`/execution`	5	Start, stop, pause, resume, status
Metrics	`/metrics`	2	Execution metrics and statistics
Audit	`/audit`	3	Audit trail query, detail, chain verification
Config	`/config`	4	Validation, settings, profile management
Connectors	`/connectors`	7	Capabilities, providers, execution, traces, governance, discovery
Contracts	`/contracts`	5	Capability contracts, artifact contracts, validation
Knowledge	`/knowledge`	4	Store, retrieve, search, delete artifacts
Evaluation	`/evaluation`	3	Run evaluations, get results, list criteria
Experiments	`/experiments`	4	Create, list, get, compare A/B tests
Benchmarks	`/benchmarks`	3	Run benchmarks, get results, manage datasets
Simulation	`/simulation`	2	Create sandbox, execute simulation run

Group	Prefix	Endpoints	Description
Events	`/events`	2	Event bus subscription, event history
Webhooks	`/webhooks`	3	Register, list, delete webhook endpoints
MCP	`/mcp`	2	MCP server status, tool listing

12.2 Common Response Patterns

All endpoints return JSON responses with consistent structure:

- **Success**: HTTP 200/201 with the resource or collection in the response body
- **Validation Error**: HTTP 422 with Pydantic validation details
- **Not Found**: HTTP 404 with `{"detail": "Resource not found"}`
- **Server Error**: HTTP 500 with `{"detail": "Internal server error"}`

List endpoints support pagination via `offset` and `limit` query parameters.

13 CLI Reference

The `agent-orchestrator` CLI provides workspace management, server control, and work submission commands.

13.1 Commands

Command	Description	Example
`init`	Initialize a new workspace	`agent-orchestrator init my-workspace`
`init --template`	Initialize from a built-in template	`agent-orchestrator init --template software-dev my-ws`
`validate`	Validate workspace configuration	`agent-orchestrator validate my-workspace`
`serve`	Start the API server	`agent-orchestrator serve my-workspace`
`submit`	Submit a work item	`agent-orchestrator submit my-ws --title "Task" --content "..."`
`agents list`	List configured agents	`agent-orchestrator agents list my-workspace`
`agents get`	Get agent details	`agent-orchestrator agents get my-ws --id analyst`
`agents create`	Create a new agent	`agent-orchestrator agents create my-ws --from agent.yaml`

Command	Description	Example
`agents delete`	Delete an agent	`agent-orchestrator agents delete my-ws --id analyst`
`status`	Show engine status	`agent-orchestrator status my-workspace`
`audit`	Query audit trail	`agent-orchestrator audit my-ws --type GOVERNANCE_DECISION`
`profiles list`	List available profiles	`agent-orchestrator profiles list my-workspace`
`profiles switch`	Switch active profile	`agent-orchestrator profiles switch my-ws --profile prod`

13.2 Global Options

Option	Description
`--verbose` / `-v`	Enable verbose logging output
`--quiet` / `-q`	Suppress non-error output
`--format` / `-f`	Output format: `text`, `json`, `table` (default: `text`)
`--config` / `-c`	Path to settings.yaml override

14 Built-in Profiles

Agent Orchestrator ships with three built-in configuration profiles that serve as starting points and reference implementations.

14.1 content-moderation

A content moderation pipeline with agents for classification, policy evaluation, and escalation.

Agent	Role	Description
Classifier	analyst	Categorizes content by type and risk level
Policy Evaluator	reviewer	Evaluates content against moderation policies
Escalation Handler	coordinator	Routes flagged content to human reviewers

Workflow: `classify` -> `evaluate` -> `decide` -> `complete` / `escalate`

14.2 software-dev

A software development pipeline with agents for requirements analysis, code generation, and code review.

Agent	Role	Description
Requirements Analyst	analyst	Decomposes requirements into implementable tasks
Code Generator	developer	Produces code based on specifications
Code Reviewer	reviewer	Reviews generated code for quality and correctness

Workflow: `analyze` -> `generate` -> `review` -> `complete` / `revise`

14.3 research-team

A research pipeline with agents for literature search, synthesis, and critical evaluation.

Agent	Role	Description
Research Scout	analyst	Searches for relevant literature and data
Synthesizer	developer	Combines findings into coherent analysis
Critical Evaluator	reviewer	Evaluates synthesis for rigor and completeness

Workflow: `search` -> `synthesize` -> `evaluate` -> `complete` / `search` (iterate)

15 Common Workflows

15.1 Workflow 1: Basic Work Item Processing

1. Initialize a workspace: `agent-orchestrator init --template software-dev my-workspace`
2. Configure API keys in `settings.yaml` or environment variables
3. Validate configuration: `agent-orchestrator validate my-workspace`
4. Start the server: `agent-orchestrator serve my-workspace`
5. Submit a work item via API or CLI
6. Monitor progress via `/api/v1/workitems/{id}` or the audit trail
7. Retrieve results from the completed work item

15.2 Workflow 2: Custom Multi-Agent Pipeline

1. Create a workspace: `agent-orchestrator init my-workspace`
2. Define agents in `profiles/default/agents.yaml` with different LLM providers
3. Design the phase pipeline in `profiles/default/workflow.yaml`
4. Set governance policies in `profiles/default/governance.yaml`
5. Validate and start the server
6. Submit work items and observe phase transitions in the audit trail

15.3 Workflow 3: Evaluation and Optimization

1. Start with a working pipeline
2. Create an A/B experiment comparing two agent configurations

3. Submit a batch of work items to generate data
4. Run LLM-as-judge evaluations against quality criteria
5. Compare experiment variants using the `/api/v1/experiments/{id}/compare` endpoint
6. Update agent configurations based on results

15.4 Workflow 4: Adding External Connectors

1. Enable connector providers by adding API keys to `settings.yaml`
2. Grant agents connector permissions in `agents.yaml`
3. Configure MCP servers in `mcp.yaml` for additional tool access
4. Run the discovery endpoint to verify available providers
5. Submit work items that exercise connector capabilities
6. Review connector traces for debugging and cost tracking

16 Troubleshooting

16.1 Server Does Not Start

Symptom: `agent-orchestrator serve` exits immediately or shows import errors.

Resolution:

- Verify Python 3.10+ is installed: `python --version`
- Ensure all dependencies are installed: `pip install "agent-orchestrator[llm]"`
- Check for port conflicts on 8000: `lsof -i :8000` (Linux/macOS) or `netstat -ano | findstr 8000` (Windows)
- Validate configuration: `agent-orchestrator validate my-workspace`

16.2 LLM Calls Failing

Symptom: Agents fail with timeout or authentication errors.

Resolution:

- Verify API keys are set correctly in environment variables or `settings.yaml`
- Check the audit trail for specific error messages: `/api/v1/audit?record_type=AGENT_ACTION`
- For Ollama, ensure the service is running and the model is pulled: `ollama list`
- Check retry policy settings – the default exponential backoff may need adjustment for rate-limited APIs

16.3 Work Items Stuck in Pending

Symptom: Submitted work items remain in `pending` status indefinitely.

Resolution:

- Verify the engine is running: `GET /api/v1/execution/status`
- Check if the engine is paused: resume with `POST /api/v1/execution/resume`
- Look for governance policies that may be blocking: `GET /api/v1/governance/reviews`
- Check `WorkQueue` depth via the metrics endpoint

16.4 Connector Invocations Failing

Symptom: Agents cannot access external systems through connectors.

Resolution:

- Verify the agent has the required connector permissions in `agents.yaml`
- Check that the provider API key is configured in `settings.yaml`
- Review connector traces: `GET /api/v1/connectors/traces`
- Ensure connector governance has not disabled the provider: `GET /api/v1/connectors/governance`
- For MCP connectors, verify the MCP server process is accessible

16.5 Hash Chain Verification Fails

Symptom: `GET /api/v1/audit/verify` reports chain integrity errors.

Resolution:

- This indicates the audit log file has been modified externally
- Identify the break point from the verification response
- Examine the JSONL file directly for manual edits or corruption
- If the file was corrupted (not tampered), restore from backup

16.6 Docker Compose Issues

Symptom: Services fail to start or cannot connect to each other.

Resolution:

- Check service logs: `docker compose logs api` and `docker compose logs db`
- Ensure `.env` file exists with required variables
- Verify PostgreSQL is healthy before the API starts: `docker compose ps`
- For Ollama container, ensure sufficient memory is allocated to Docker
- Reset state if needed: `docker compose down -v && docker compose up -d`